Presentation script: CacheWise replication and KV cache recovery

This script follows the eleven-slide deck and its 10:00 timing plan. Spend 0:00–5:00 on the CacheWise paper and adapted replication (slides 1–6), then 5:00–10:00 on recovery costs, host methods, limitations and takeaways (slides 7–11). Questions follow the talk. Read the spoken paragraphs; italic instructions are delivery cues and should not be read aloud. Pause briefly after the main numbers and use the slide transitions to check your timing.

Slide 1 — CacheWise replication and KV cache recovery

0:00–0:20 · 20 seconds

Introduce the project and its central question. Move to the CacheWise paper at twenty seconds.

Hello everyone. This project asks how coding-agent servers can reuse previous computation when GPU memory is limited.

We examine CacheWise scheduling and eviction, then look at restoring evicted KV state from host memory.

Slide 2 — What do the CacheWise traces contain?

0:20–1:05 · 45 seconds

Point to the logged events, then the reuse patterns and the scheduling explanation.

CacheWise uses CATraces: logs of real Claude Code sessions. They record model requests, tool calls, timing information, and token usage.

These are multi-turn sessions with growing prompts. Successive requests repeatedly share long prefixes, while generated outputs are relatively short. Tool waits separate the requests.

The released content is redacted, so our replay uses synthetic tokens to preserve lengths and prefix growth.

When a prefix is already on the GPU, serving that request requires less new prefill and fewer additional KV blocks. Prefix-aware ordering can therefore reduce the active working set and eviction pressure.

Slide 3 — CacheWise: scheduling and eviction

1:05–1:55 · 50 seconds

Point to scheduling first, then eviction. Connect both decisions back to prefix reuse.

These trace properties motivate two CacheWise mechanisms.

First, prefix-aware scheduling favors waiting requests needing fewer additional KV blocks, using overlap with state already on the GPU.

Second, predictive eviction uses tool metadata to estimate when inactive sessions will return and guide which cached state to retain.

Scheduling decides what to serve. Eviction decides what to reclaim. They interact because request ordering changes cache residency and future reuse.

Our replication tests the two decisions independently. The later host study then examines how to recover useful state after eviction.

Slide 4 — Replication separates two decisions

1:55–2:55 · 60 seconds

Point to the policy matrix and the coverage-study box.

We compare first-come, first-served with prefix-aware scheduling, and least recently used eviction, or LRU, with predictive eviction.

All four combinations use the same configuration, fresh servers, and identical selected inputs. The replay preserves recorded lengths and within-session prefix growth.

The setup uses a fourteen-billion-parameter Qwen coding model on one Blackwell GPU, with about 288,000 KV token slots.

Cohort A tests nested loads of ten, twenty, and thirty segments. B and C check whether the ordering holds on different held-out sessions. All are evaluation cohorts.

Five settings, four policies, and three repeats give sixty runs and 6,384 request executions. Our main metric sums model-request latencies within a session, excluding tool waits.

Slide 5 — Prefix / LRU has the lowest mean

2:55–4:05 · 70 seconds

Trace the green line. Pause after “all five settings.”

This graph shows cohort A as the selected workload grows from ten to thirty concurrent segments. Lower is better.

The blue and orange lines use first-come, first-served scheduling. The green and purple lines use prefix-aware scheduling.

The largest improvement comes from changing the scheduler. Prefix-aware LRU lowers the mean by about 58 percent at ten segments, 62 percent at twenty, and 65 percent at thirty, compared with first-come, first-served LRU.

The report also includes the additional B and C cohorts. Prefix-aware LRU has the lowest mean in all five settings.

The error bars show the observed range of three repeats. They are not confidence intervals. Also, increasing the load adds new segments, so workload composition changes too.

The result supports prefix-aware scheduling in this replay. Next, we can ask whether predictive eviction adds to that benefit.

Slide 6 — Prediction depends on the scheduler

4:05–5:00 · 55 seconds

Explain the workload key, then point below zero for orange and above zero for purple.

A, B, and C are workload groups. A tests ten, twenty, and thirty concurrent segments, with larger sets retaining the smaller ones.

B and C each contain ten segments from different held-out source sessions, disjoint from A and one another. They check whether the result depends on the chosen workload. N means concurrent segments.

Each dot compares predictive eviction with LRU within one repeat. Below zero means prediction is faster.

Every orange point is below zero; every purple point is above zero. Prediction helps first-come, first-served but makes prefix-aware scheduling slower across these workloads.

Lower session means also do not guarantee better request tails. Next is recovery after eviction.

Slide 7 — Preserve evicted KV in host DRAM

5:00–6:00 · 60 seconds

Start this section at 5:00. Follow the save arrow to host memory, then the restore arrow back to the GPU.

I'll now move to the separate Mini-SGLang host-recovery study.

Instead of discarding every evicted prefix, the implementation can save its KV pages in pinned host memory. A shared prefix index tracks whether pages are on the GPU or in the host cache.

When a request returns, the engine restores a compatible host copy before prefill. If no copy survives, it recomputes.

Clean host copies can be retained, avoiding another save when those GPU pages are evicted again. Predictive prefetch can also start restoration before a session returns.

This study keeps twenty-four gibibytes of GPU KV capacity and adds thirty-six gibibytes of host KV capacity. The implemented backing tier is host DRAM; SSD storage remains future work.

Slide 8 — Host restoration reduces session time

6:00–7:15 · 75 seconds

Read the cost condition, then point to each pair of bars. Pause after each ratio.

The cost condition is straightforward: saving, restoring, and managing a copy should cost less than recomputing the missing prefix.

Here, scheduling and eviction stay fixed at first-come, first-served and LRU. We change the recovery method.

At twenty-eight concurrent episodes, reported mean session time falls from 821 to 90.7 seconds, a ratio of about 9.1. At thirty-six episodes, it falls from 1,060 to 222 seconds, a ratio of about 4.8.

The reported missing-prefix deficit also decreases, consistent with more reusable state surviving.

These comparisons include the extra host capacity, and each bar represents one reported run. They show favorable host-memory operating points rather than an equal-total-memory policy comparison.

This study also uses a different engine and replay setup, so we keep its timings separate from the replication results.

Slide 9 — Fetch-aware ordering discounts host hits

7:15–8:35 · 80 seconds

Point to the priority cost, then compare the prefetch and fetch-aware bars.

A request with a host copy is different from a complete cache miss. Fetch-aware scheduling makes that distinction explicit.

Its priority combines tokens needing new prefill with a discounted cost for tokens restored from the host. The discount factor is 0.05, so lower estimated recovery cost goes first. This is a heuristic rather than a measured latency predictor.

At thirty-six episodes, predictive eviction with prefix scheduling records 158 seconds. Adding prefetch alone records 172 seconds. The full fetch-aware combination records 136 seconds.

The diamonds show additional observations: 175 seconds for the predictive-prefix control and 135 for the full combination. The two descriptive means are 166.5 and 135.5 seconds.

These observations favor the full combination, but there are too few repetitions to establish statistical significance or isolate every component.

The design lesson is to consider cache location when ordering requests. Prefetching alone does not guarantee an improvement.

Slide 10 — Keep the conclusions within the evidence

8:35–9:20 · 45 seconds

Move across the three limitation cards without reading every line.

Three limits frame these findings.

First, the replication uses synthetic content and selected workloads. One preemption violated the original audit rule. We retained it under a documented post hoc amendment; its extra computation is unmeasured.

Second, the host study uses a separate replay and adds memory capacity. Most configurations have one reported observation, so its policy comparisons are descriptive.

Third, we have no SSD implementation or measurements. An SSD extension would need storage reads and writes, bounded staging, compatibility metadata, and failure handling.

The completed evidence therefore concerns this adapted replication and host-DRAM recovery.

Slide 11 — Three takeaways

9:20–10:00 · 40 seconds

Deliver each takeaway directly to the audience. Leave a short pause before inviting questions.

To conclude, there are three takeaways.

First, scheduling and eviction interact. Prefix-aware LRU has the lowest mean in all five replication settings, while predictive eviction changes direction with the scheduler.

Second, saving KV state in host memory can avoid costly recovery, with large reported gains when backing capacity is added.

Third, request ordering should account for where reusable state resides. Fetch-aware scheduling is promising here, while SSD evaluation remains future work.

Thank you. I’m happy to take questions.